SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	N.J.Sreelakshmi
Register Number	1920260072
Course Code & Name	CSA0209 - C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co- operative
GitHub Repository URL	https://github.com/sreelakshminj6825-ctrl/co1.git
Submission Date	27/10/2026

TASK 1 — Understand the Problem and Identify the Data

Task / Problem Statement

A rural electricity co-operative bills households every month from two meter readings. The program must be designed so that every item of data it stores and computes is identified and justified before any code is written. This task analyses the billing problem: its inputs and outputs, the values that never change, the values that can be negative, fractional or very large, the rules every correct bill must satisfy, and the assumptions made. It also analyses the alternative of reading meter readings as text (e.g. "01530") instead of integers.

Aim

To analyse the billing problem (Bloom's level K4) and identify and justify every data item, constant, rule and assumption, and to compare integer and text storage of meter readings in time and space.

Concept / Theory

A program should be designed from its data. For each item we ask: is it an input (supplied by the user), an output (computed), or a constant (fixed for the whole run)? The answer decides the C data type, whether it is declared as a constant, and what validation is needed. Three ideas are used here:

- Invariants / rules: statements that must be true for every correct bill, such as "a bill is never negative". They become test conditions.
- Number representation: meter readings are whole-number counters, but the tariff (Rs 2.25 per unit) is fractional in rupees. Fractions cause rounding error in binary floating point, so money is better held as whole paise.
- Time and space complexity: an integer comparison takes O(1) time and a fixed number of bytes, whereas a text reading with d digits needs O(d) time to compare or convert and d+1 bytes of storage.

Procedure / Algorithm

1. Read the problem statement and list every value that enters the program (inputs) and every value it must produce (outputs).
2. Mark values that never change during a run (tariff slabs, rates, fixed charge, register size) as constants.
3. For each item decide whether it can be negative, fractional, or very large, and choose a representation (whole paise, unsigned counters).
4. Write the rules (invariants) that every bill must satisfy and the assumptions made to keep the problem well defined.
5. Write a small C program that stores the data items, computes one bill, checks the rules, and compares integer and text readings; run it and study the output.

Data Analysis — Inputs, Outputs and Justification

Data item	Role	Justification	Proposed C type
Household number	Input	Identifies the consumer. Never used in arithmetic and never negative, so it is an identifier, not a quantity. A 6–7 digit number fits easily in 32 bits.	unsigned int
Category	Input	A single code (D = domestic, C = commercial, I = industrial). One character is enough and it is never calculated with.	char
Previous reading	Input	Value shown on the 16-bit meter register at the last visit. A whole, non-negative counter in the range 0 – 65535.	unsigned short (meter_t)
Current reading	Input	Value on the register now. Same range as the previous reading; may be smaller than it after a rollover.	unsigned short (meter_t)
Units consumed	Output	(current - previous) modulo 65536. Always a whole number ≥ 0, so unsigned arithmetic is correct.	unsigned int
Energy charge	Output	Slab-wise charge. Rates are Rs 2.25 and Rs 4.50, so the value is fractional in rupees but a whole number in paise.	long (paise)
Fixed charge	Output / constant	Rs 50 for every bill, even with 0 units. Held as 5000 paise.	long (paise)
Total bill	Output	Energy charge + fixed charge. Summed over a district it	long (paise)

Constants (values that never change during a run)

- Slab limits: 100 units and 200 units. They define where the rate changes, so they must be named once and reused.
- Rates: 0 paise (first slab), 225 paise (second slab), 450 paise (above 200 units). A tariff revision changes only these.
- Fixed charge: 5000 paise (Rs 50).
- Register size: the meter counts 0 – 65535 and then rolls over, so the modulus 65536 (or the maximum 65535) is a constant of the hardware.
- Paise per rupee: 100, used only when printing a bill as rupees.paise.

Values that can be negative, fractional or very large

- Negative: no output may be negative. However, the raw difference current - previous is negative after a rollover (for example 120 - 65400 = -65280). This is why unsigned 16-bit arithmetic is used instead of signed subtraction.
- Fractional: the rate Rs 2.25 per unit is fractional in rupees. Storing money as whole paise (225) removes every fraction. Meter readings themselves are whole units.
- Very large: one household's bill is small, but a district total is not. For 100,000 households at about Rs 1,000 each the total is 10^10 paise, which is more than the largest 32-bit int (2,147,483,647). A 64-bit integer is therefore used for money totals.

Rules every correct bill must satisfy

- The bill is never negative: energy charge ≥ 0, fixed charge ≥ 0, total ≥ 0.
- Total = energy charge + fixed charge (the total equals the sum of its parts).
- Units consumed are never negative and lie between 0 and 65535 for one billing period.
- A household that uses 0 units still pays the fixed charge, so total ≥ 5000 paise.
- The bill never decreases when units increase (monotonic) and is exact to the paisa (an integer number of paise).

Assumptions

- Meters are read once per billing period and the register rolls over at most once between two readings.
- Units are whole numbers; the meter has no fractional part.
- The tariff in the question applies to every category; the category is stored and printed but does not change the rates.
- Readings are validated before use (digits only, within 0 – 65535) and the fixed charge is applied to every bill.

Program / Source Code

task1.c — data items, rule check, and integer vs text readings

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <limits.h>

/* Task 1: data items of the billing problem, and integer vs text meter readings */

#define FIXED_CHARGE_PAISE 5000L

/* Rule checker: every correct bill must satisfy these */
int bill_is_valid(long energy, long fixed, long total, unsigned units)
{
    return energy >= 0 && fixed >= 0 && total >= 0 && total == energy + fixed && units < 100000u;
}

/* Text representation: compare two zero-padded digit strings, O(d) */
int text_compare(const char *a, const char *b, int *steps)
{
    int i = 0;
    while (a[i] && b[i]) { (*steps)++; if (a[i] != b[i]) return a[i] < b[i] ? -1 : 1; i++; }
    return 0;
}

/* Text representation: convert digit string to integer, O(d) */
unsigned text_to_uint(const char *s, int *steps)
```

```
{
    unsigned v = 0;
    for (int i = 0; s[i]; i++) { (*steps)++; v = v * 10 + (unsigned)(s[i] - '0'); }
    return v;
}

int main(void)
{
    /* --- Data items and the C types proposed for them --- */
    unsigned short prev = 1200, curr = 1530;   /* meter readings   */
    int household_no = 4071;                   /* identifier       */
    char category = 'D';                       /* D=domestic etc.  */
    unsigned units = (unsigned short)(curr - prev);
    long energy = 0, fixed = FIXED_CHARGE_PAISE;  /* paise     */
    long total;

    /* sample slab calculation (same tariff as Task 3) */
    if (units > 200) energy += (long)(units - 200) * 450;
    if (units > 100) energy += (long)((units > 200 ? 200 : units) - 100) * 225;
    total = energy + fixed;

    printf("Household %d  Category %c\n", household_no, category);
    printf("Previous=%hu  Current=%hu  Units=%u\n", prev, curr, units);
    printf("Energy=%ld paise Fixed=%ld paise  Total=%ld paise (Rs %ld.%02ld)\n",
           energy, fixed, total, total / 100, total %100);
    printf("Rule check (bill never negative, total = parts): %s\n\n",
           bill_is_valid(energy, fixed, total, units) ? "PASS" : "FAIL");

    /* --- Alternative: readings stored as text --- */
    const char *tprev = "01530", *tcurr = "00120";
    int steps_cmp = 0, steps_conv = 0;
    int c = text_compare(tprev, tcurr, &steps_cmp);
    unsigned a = text_to_uint(tprev, &steps_conv);
    unsigned b = text_to_uint(tcurr, &steps_conv);

    printf("Text readings \"%s\" and \"%s\"\n", tprev, tcurr);
    printf("  compare result = %d  (digit steps used = %d, d = %zu)\n", c, steps_cmp, strlen(tprev));
    printf("  converted to int: %u and %u (digit steps used = %d = 2d)\n", a, b, steps_conv);
    printf("Space per reading: int/short = %zu bytes, text = %zu bytes (d+1 chars incl. '\\0')\n",
           sizeof(unsigned short), strlen(tprev) + 1);
    return 0;
}
```

Output Screenshot

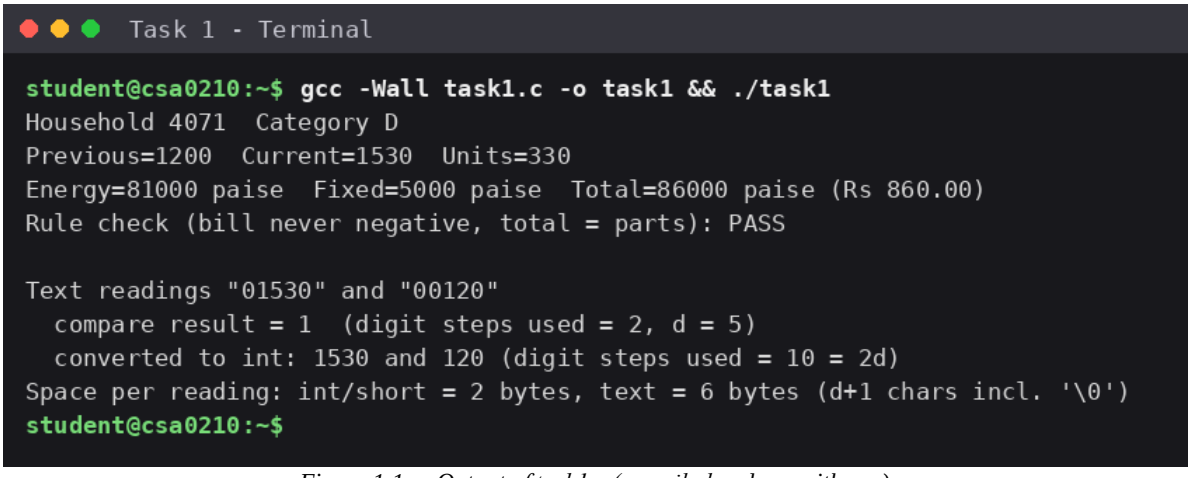


Figure 1.1 — Output of task1.c (compiled and run with gcc).

Analysis / Discussion

The output shows 330 units for readings 1200 → 1530. The energy charge is 100 free units, 100 × 225 = 22,500 paise and 130 × 450 = 58,500 paise, giving 81,000 paise. Adding the fixed charge of 5,000 paise gives 86,000 paise = Rs 860.00. The rule check printed PASS, because the bill is not negative and the total equals energy + fixed.

Alternative concept — readings stored as text. If a reading such as "01530" is held as a character string, the program needs an extra step before any arithmetic: each string must be validated and converted to a number. Let d be the number of digits.

Aspect	Integer (unsigned short)	Text (char[d+1])
Compare two readings	One machine comparison: O(1)	Digit by digit until a difference: O(d) (equal length, zero padded)
Convert to a number	Not needed	One multiply-and-add per digit: O(d) per reading, so 2d steps for two readings (10 in the output for d = 5)
Subtract / roll-over fix	One unsigned subtraction: O(1)	Convert both, then subtract: O(d) in total
Space per reading	2 bytes, fixed	d + 1 bytes (6 bytes for "01530", including the terminating '\0')
Validation	Range is guaranteed by the type	Must check every character is a digit and the length is correct
Advantage	Fast, small	Keeps leading zeros exactly as printed on the meter

For a few thousand households d = 5 is tiny, so the extra time is not noticeable, but the integer form is still simpler, smaller and safer. The sample run confirms the analysis: comparison used 2 digit steps before finding a difference, conversion used 2d = 10 steps, and the text needs 6 bytes against 2 bytes.

Result

All inputs, outputs, constants, rules and assumptions of the billing problem were identified and justified, and the integer representation of meter readings was shown to be better than text in time (O(1) against O(d)) and space (2 bytes against d + 1). The main lesson is that the rollover case makes a raw signed difference unsafe and that money should be held in whole paise.

TASK 2 — Choose the Data Types

Task / Problem Statement

Select a C data type for every data item identified in Task 1, check its size with sizeof and its range with limits.h and float.h, explain why money must not be stored as float rupees, show a case where float gives a wrong total for 4,000 bills, and complete TODO 1 of consumption() without if–else so that it is correct when a 16-bit meter register rolls over past 65535.

Aim

To choose suitable C data types with justification, to demonstrate a floating-point error and an overflow/rollover case, and to implement the rollover-safe consumption() function.

Concept / Theory

Every C type has a fixed size and range that depend on the system, found with sizeof, limits.h (integers) and float.h (floating point). Unsigned integers do not overflow in the undefined sense: they wrap around modulo 2^n, so an unsigned 16-bit value behaves exactly like a meter register that returns to 0 after 65535.

float and double store numbers in binary with a limited number of significant digits (about 6–7 for float and 15–16 for double). Values such as 0.1 or 123.45 have no exact binary form, so each addition carries a small error and these errors accumulate. Money must be exact, so it is kept as an integer number of paise.

Procedure / Algorithm

1. For each data item from Task 1 choose the smallest type that safely holds its full range, and print its size and range using sizeof, limits.h and float.h.
2. Define meter_t as unsigned short, because the register is 16 bits wide.
3. In consumption(), subtract the readings and cast the result back to meter_t: (meter_t)(current - previous). This reduces the answer modulo 65536, so no if–else is needed.
4. Run main() with the readings (1200, 1530) and (65400, 120) and compare with the expected output.

5. Add 4,000 bills of Rs 123.45 using float, double and whole paise, and compare each total with the exact value Rs 493,800.00.

Data-Type Table

Data item	Chosen type	Size (this system)	Range	Reason
Household number	unsigned int	4 bytes	0 to 4,294,967,295	Whole, non-negative; far larger than any district size
Category	char	1 byte	-128 to 127	One code character
Previous / current reading	unsigned short (meter_t)	2 bytes	0 to 65,535	Matches the 16-bit register exactly
Units consumed	unsigned int	4 bytes	0 to 4,294,967,295	Whole units; holds one bill easily
Energy, fixed charge, total (paise)	long	8 bytes	±9.22 × 10^18	Exact integer money; no overflow even for a district
District total units	unsigned long / long long	8 bytes	up to 1.8 × 10^19 / 9.22 × 10^18	100,000 × 65,535 ≈ 6.5 × 10^9 exceeds unsigned int
Money as rupees (rejected)	float / double	4 / 8 bytes	±3.4 × 10^38 / ±1.8 × 10^308	Inexact: only ~6 / ~15 significant digits

A district of 100,000 households can be summed without overflow only if the sum uses a 64-bit type; a 32-bit int (maximum 2,147,483,647) is too small, so the range of int alone is not sufficient for a district total. (On some systems, e.g. 64-bit Windows, long is only 32 bits, so long long is the portable choice.)

Program / Source Code

task2.c — consumption() with TODO 1 completed

```
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
   // TODO 1: Return the units consumed, correct even when the register has rolled over
   // (current < previous); use unsigned 16-bit arithmetic instead of an if-else
   return (meter_t)(current - previous);
}

int main(void)
{
   printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
        sizeof(meter_t), (unsigned)USHRT_MAX);
   printf("Consumption: %u units\ n", consumption(1200, 1530));
   printf("Consumption: %u units\n", consumption(65400, 120)); /* rolled over */
   return 0;
}
```

task2_types.c — type sizes, ranges and the float error demonstration

```
#include  <stdio.h>
#include  <limits.h>
#include <float.h>

int main(void)
{
   printf("%-22s %-6s %-28s\n", "Type", "Bytes", "Range");
   printf("%-22s %-6zu %d to %d\n", "char", sizeof(char), CHAR_MIN, CHAR_MAX);
   printf("%-22s %-6zu %d to %d\n", "short", sizeof(short), SHRT_MIN, SHRT_MAX);
   printf("%-22s %-6zu 0 to %u\n", "unsigned short", sizeof(unsigned short), (unsigned)USHRT_MAX);
   printf("%-22s %-6zu %d to %d\n", "int", sizeof(int), INT_MIN, INT_MAX);
   printf("%-22s %-6zu 0 to %u\n", "unsigned int", sizeof(unsigned), UINT_MAX);
```

```
    printf("%-22s %-6zu %ld to %ld\n", "long", sizeof(long), LONG_MIN, LONG_MAX);
    printf("%-22s %-6zu %lld to %lld\n", "long long", sizeof(long long), LLONG_MIN, LLONG_MAX);
    printf("%-22s %-6zu +/-%e (~%d digits)\n", "float", sizeof(float), FLT_MAX, FLT_DIG);
    printf("%-22s %-6zu +/-%e (~%d digits)\n", "double", sizeof(double), DBL_MAX, DBL_DIG);

    /* Adding 4,000 bills of Rs 123.45 */
    float  fsum = 0.0f;
    double dsum = 0.0;
    long   psum = 0;                /* paise */
    for (int i = 0; i < 4000; i++) { fsum += 123.45f; dsum += 123.45; psum += 12345L; }

    printf("\nAdding 4000 bills of Rs 123.45 (exact total Rs 493800.00)\n");
    printf("float  total : Rs %.4f   error = %+.4f\n", fsum, (double)fsum - 493800.0);
    printf("double total : Rs %.10f  error = %+.3e\n", dsum, dsum - 493800.0);
    printf("long paise  : %ld paise = Rs %ld.%02ld  error = 0\n", psum, psum/100, psum%100);

    /* error growth with n */
    printf("\n%-10s %-16s %-16s\n", "n bills", "float error", "double error");
    int ns[] = {100, 1000, 4000, 100000};
    for (int k = 0; k < 4; k++) {
        float f = 0; double d = 0;
        for (int i = 0; i < ns[k]; i++) { f += 123.45f; d += 123.45; }
        double exact = ns[k] * 123.45;
        printf("%-10d %-16.6f %-16.3e\n", ns[k], (double)f - exact, d - exact);
    }
    return 0;
}
```

Output Screenshot

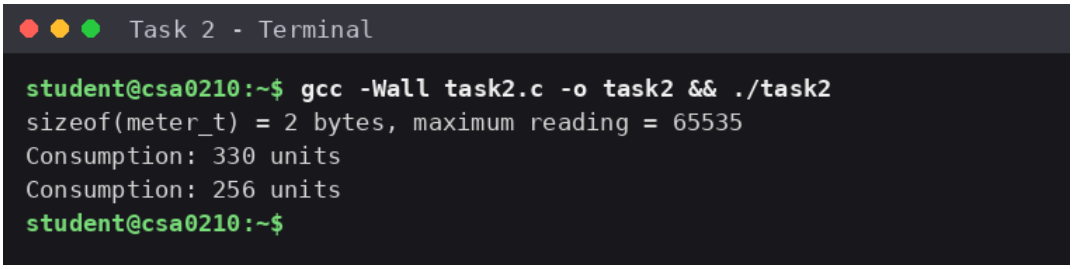


Figure 2.1 — Output of task2.c (matches the expected output: 330 and 256).

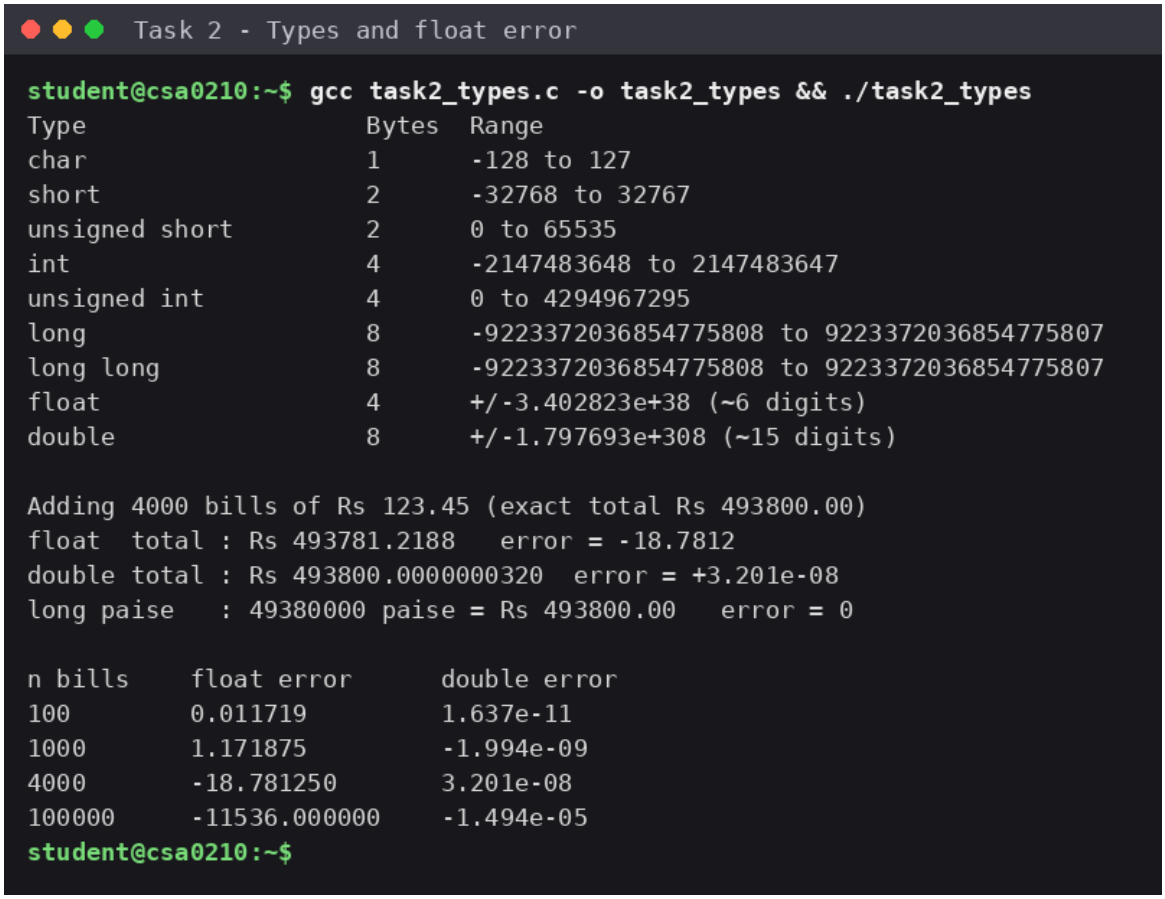


Figure 2.2 — Output of ta3k2_type3.c: 3izeof/limit3.h/float.h table and the 4,000-bill float error.

Analy3i3 / Di3cu33ion

Why the 3econd re3ult i3 256 and not a negative number. In con3umption(65400, 120) the expre33ion current – previou3 i3 fir3t promoted to int, giving 120 – 65400 = -65280. Ca3ting that value to meter_t (un3igned 3hort) convert3 it modulo 2^16 = 65536, and -65280 + 65536 = 256. Thi3 i3 correct: the regi3ter counted 136 unit3 from 65400 up to it3 rollover at 65536 and then 120 more unit3, 3o 136 + 120 = 256. An un3igned type cannot hold a negative number, 3o wrapping (not a negative value) i3 the defined re3ult.

Why not float rupee3. An un3igned 3hort fail3 by wrapping around, but a float fail3 in a different way: it cannot repre3ent 123.45 exactly, and above about 500,000 the gap between neighbouring float3 i3 0.03125, 3o 3mall amount3 are rounded away. In Figure 2.2, adding 4,000 bill3 of R3 123.45 3hould give R3 493,800.00 but float give3 R3 493,781.2188 , an error of R3 18.78 . Double i3 much clo3er (error about 3 × 10^-8) but 3till not exact. Storing 12,345 pai3e per bill a3 a long give3 exactly 49,380,000 pai3e = R3 493,800.00 with zero error.

Alternative concept — double rupee3 in3tead of long pai3e. The bill could be kept a3 a double and rounded to two decimal3 when printed or 3tored (for example llround(rupee3 × 100) / 100.0), and the rounding would have to be applied to every bill, not ju3t the final total, otherwi3e accumulated error can 3how up a3 a wrong pai3a. Compari3on:

A3pect	long pai3e	double rupee3
Time per addition	O(1)	O(1)
Space per value	8 byte3 here (long)	8 byte3 (double)
Where rounding i3 applied	Nowhere — all value3 are exact integer3	On every bill (and when printing), to the neare3t 0.01
Error after n addition3	0 for every n	Grow3 with n: wor3t ca3e about n × ε (ε ≈ 1.1 × 10^-16 relative); in the run 1.6 × 10^-11 (n = 100) to 1.5 × 10^-5 (n = 100,000)
float for compari3on	—	Error 0.0117 (n = 100), 1.17 (n = 1,000), 18.78 (n = 4,000), 11,536 (n = 100,000)

Both approaches take O(1) time per addition and the same space on this system, so the deciding factor is correctness: integer paise is exact, whereas floating point needs rounding rules and the error grows as more bills are added.

Result

A justified data-type table with sizes and ranges was produced. consumption() was completed with a single unsigned 16-bit expression and gives the expected 330 and 256 units. The float error (₹ 18.78 over 4,000 bills) was demonstrated and was resolved by storing money as whole paise in a long.

TASK 3 — Define the Constants

Task / Problem Statement

Define the tariff values and limits as constants so that a tariff revision is made in one place only, and complete compute_bill() using only the macros defined at the top. The function returns the bill in paise: the fixed charge plus the energy charge calculated slab by slab (first 100 units free, units 101–200 at ₹ 2.25, units above 200 at ₹ 4.50).

Aim

To use #define, const and enum appropriately, to apply named constants in a slab-wise bill calculation, and to show that changing one constant changes every bill consistently.

Concept / Theory

#define is a preprocessor macro: the compiler never sees the name, because the preprocessor replaces it with the text before compilation. It has no type, no storage and no scope (it is valid from its definition to the end of the file). const creates a real, typed variable that obeys scope rules, can be seen in the debugger, and is type checked, but it normally takes storage and in C is not a constant expression (it cannot be used as an array size in older C). enum creates named integer constants that are known at compile time, are visible to the debugger, and are good for related whole-number values such as category codes.

	#define	const variable	enum
Type checking	None (text substitution)	Yes, has a declared type	Always int
Scope	From definition to end of file	Block / file scope	Block / file scope
Debugging	Name not visible to debugger	Visible, can inspect value	Visible as a name
Storage	None	May occupy memory	None (compile-time value)
Best used for	Tariff numbers, limits, literal suffixes like 5000L	Typed values e.g. double constants	Groups of related integers e.g. categories

Naming conventions: constants are written in UPPER_CASE with underscores, grouped by prefix and ending in the unit, for example SLAB1_UNITS, RATE2_PAISE and FIXED_CHARGE_PAISE. The unit in the name prevents mixing paise with rupees.

Procedure / Algorithm

1. Define the slab limits, the three rates and the fixed charge as macros at the top of the file.
2. Start the bill with paise = FIXED_CHARGE_PAISE.
3. If units > SLAB2_UNITS, add (units - SLAB2_UNITS) × RATE3_PAISE.
4. If units > SLAB1_UNITS, add the units that fall between SLAB1_UNITS + 1 and SLAB2_UNITS (at most SLAB2_UNITS − SLAB1_UNITS) × RATE2_PAISE.
5. Add the remaining units (at most SLAB1_UNITS) × RATE1_PAISE, return paise, and print each bill as ₹ rupees.paise using p / 100 and p % 100. Then change RATE2_PAISE to 230 and run again.

Program / Source Code

task3.c — compute_bill() using only the macros (RATE2_PAISE = 225)

```
#include <stdio.h>

#define SLAB1_UNITS 100 /* first 100 units */
#define SLAB2_UNITS 200 /* units 101 to 200 */
```

```c
#define RATE1_PAISE 0 /* first slab is free */
#define RATE2_PAISE 225 /* Rs 2.25 per unit */
#define RATE3_PAISE 450 /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L /* Rs 50 fixed charge */

/* Returns the bill in paise, so that no rounding error can occur. */
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    // TODO 1: slab by slab using the macros
    if (units > SLAB2_UNITS)
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    if (units > SLAB1_UNITS) {
        int in_slab2 = (units > SLAB2_UNITS ? SLAB2_UNITS : units) - SLAB1_UNITS;
        paise += (long)in_slab2 * RATE2_PAISE;
    }
    paise += (long)(units < SLAB1_UNITS ? (units > 0 ? units : 0) : SLAB1_UNITS) * RATE1_PAISE;
    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Output Screenshot

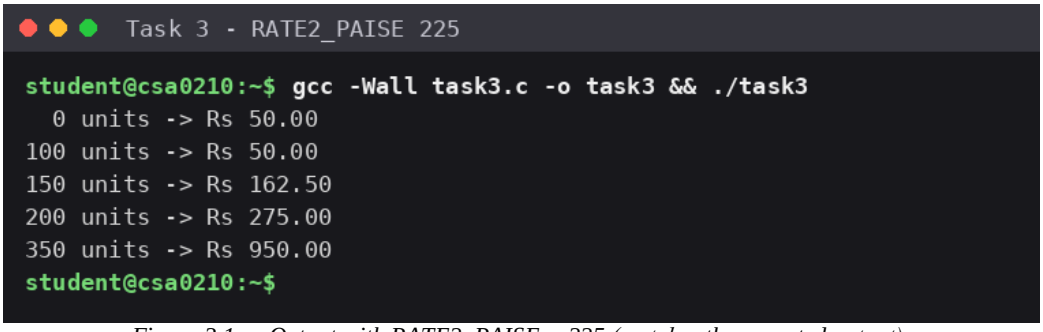


Figure 3.1 — Output with RATE2_PAISE = 225 (matches the expected output).

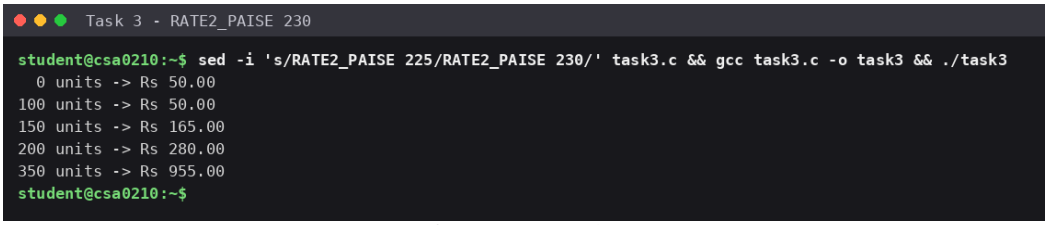


Figure 3.2 — Output after changing only RATE2_PAISE to 230.

Analysis / Discussion

With RATE2_PAISE = 225 the five test readings give Rs 50.00, 50.00, 162.50, 275.00 and 950.00, exactly the expected output. For example, 150 units = 5,000 + 50 × 225 = 16,250 paise = Rs 162.50, and 350 units = 5,000 + 100 × 225 + 150 × 450 = 95,000 paise = Rs 950.00.

After changing the single line RATE2_PAISE to 230 every affected bill changed consistently: 150 units rose to Rs 165.00 (+50 × 5 paise = Rs 2.50), 200 units to Rs 280.00 and 350 units to Rs 955.00 (+100 × 5 paise = Rs 5.00 each), while 0 and 100 units stayed at Rs 50.00 because no unit lies in the second slab. No other line of code had to be edited, which is the purpose of named constants.

Alternative concept — const variables or an enum instead of #define.

- What would change: const int SLAB1_UNITS = 100; (and similar) would replace the macros. The code in compute_bill() stays the same, but the constants become typed objects. FIXED_CHARGE_PAISE would be declared const long so that its long type is kept; with an enum every value would be an int, so a cast such as (long) is needed before multiplying to avoid int overflow.
- Type checking: const gives a real type and compiler warnings for misuse; #define has none; enum values are always int.
- Debugger visibility: const variables and enum names can be inspected in a debugger, while a #define name disappears after preprocessing.
- Memory: a #define occupies no storage and an enum constant also none; a const variable may occupy a few bytes (it is often optimised away with optimisation on).
- Running time: unchanged, O(1) per bill. At most a const variable is loaded from memory instead of being an immediate operand, which is negligible.

Choice made: #define for the tariff numbers and limits (they are pure compile-time numbers, and the L suffix on 5000L fixes the long type); an enum would be used for the category codes (D, C, I); a const variable would be used for a typed value such as a double conversion factor.

Result

The tariff was defined as one named set of constants, compute_bill() was completed using only those macros, and all expected outputs were obtained. Changing RATE2_PAISE to 230 changed every bill that uses the second slab consistently, proving that a tariff revision needs only one edit.

TASK 4 — Use Macros Safely

Task / Problem Statement

Write macros for small repeated calculations (converting rupees to paise and finding the larger of two values), show how the preprocessor expands them with gcc -E, demonstrate why SQUARE(x) written as x*x fails for SQUARE(a+1) and why MAX(a, b) can evaluate an argument twice (MAX(i++, j)), and state when a function is a better choice than a macro.

Aim

To write correct, fully parenthesised macros, to prove their expansion with gcc -E, and to understand the pitfalls of macros compared with static inline functions.

Concept / Theory

A macro is expanded by the preprocessor before compilation: it copies the argument text into the macro body with no understanding of types or precedence. The command gcc -E stops after this stage and prints the expanded source, so the exact text that the compiler will receive can be examined. Two classic pitfalls follow from pure text substitution:

- Precedence: SQUARE(x) defined as x * x turns SQUARE(a+1) into a+1 * a+1, which is evaluated as a + (1*a) + 1. Parentheses around each argument and around the whole body fix it: ((x) * (x)).
- Double evaluation: MAX(a, b) as (((a) > (b)) ? (a) : (b)) writes each argument twice, so MAX(i++, j) can execute i++ twice (a side effect repeated).

A function is the better choice when arguments have side effects, when type checking is wanted, when the logic needs several statements or local variables, when recursion or a function address is needed, or when the code should be easy to debug and step through. A macro is acceptable for tiny, side-effect-free, type-independent expressions, or when features such as stringising are needed.

Procedure / Algorithm

1. Define RUPEES_TO_PAISE(r) as ((r) * 100L), and define a deliberately wrong SQUARE_BAD(x) as x * x for comparison.
2. Define the correct macros SQUARE(x) as ((x) * (x)) and MAX(a, b) as (((a) > (b)) ? (a) : (b)).
3. Define static inline functions square_fn() and max_fn() as the alternative.
4. In main(), call each version with a = 4 for SQUARE(a+1), and with i = 6, j = 5 for MAX(i++, j) and max_fn(i++, j); print the results and the final value of i.
5. Run gcc -E -P task4.c to display the expanded code of main() and compare the expansions with the printed

results.

Program / Source Code

task4.c — macros, their pitfalls, and static inline alternatives

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r)  ((r) * 100L)

/* BAD version (pitfall): no parentheses around x */
#define SQUARE_BAD(x)   x * x

/* GOOD versions: every argument and the whole result in parentheses */
#define SQUARE(x)     ((x) * (x))
#define MAX(a, b)       (((a) > (b)) ? (a) : (b))

/* Alternative: static inline functions (type-checked, arguments evaluated once) */
static inline int  square_fn(int x)        { return x * x; }
static inline int  max_fn(int a, int b)    { return a > b ? a : b; }

int main(void)
{
    int a = 4;
    printf("Rs 50 in paise            : %ld\n", RUPEES_TO_PAISE(50));
    printf("SQUARE_BAD(a+1), a=4      : %d  (expected 25)\n", SQUARE_BAD(a+1));
    printf("SQUARE(a+1),   a=4        : %d   (correct)\n", SQUARE(a+1));

    int i = 6, j = 5;
    int m = MAX(i++, j);
    printf("MAX(i++, j) macro, i=6,j=5 : result=%d i=%d  (i incremented twice)\n", m, i);

    i = 6; j = 5;
    m = max_fn(i++, j);
    printf("max_fn(i++, j) inline       : result=%d i=%d (i incremented once)\n", m, i);
    printf("square_fn(a+1)  inline      : %d\n", square_fn(a+1));
    return 0;
}
```

Output Screenshot

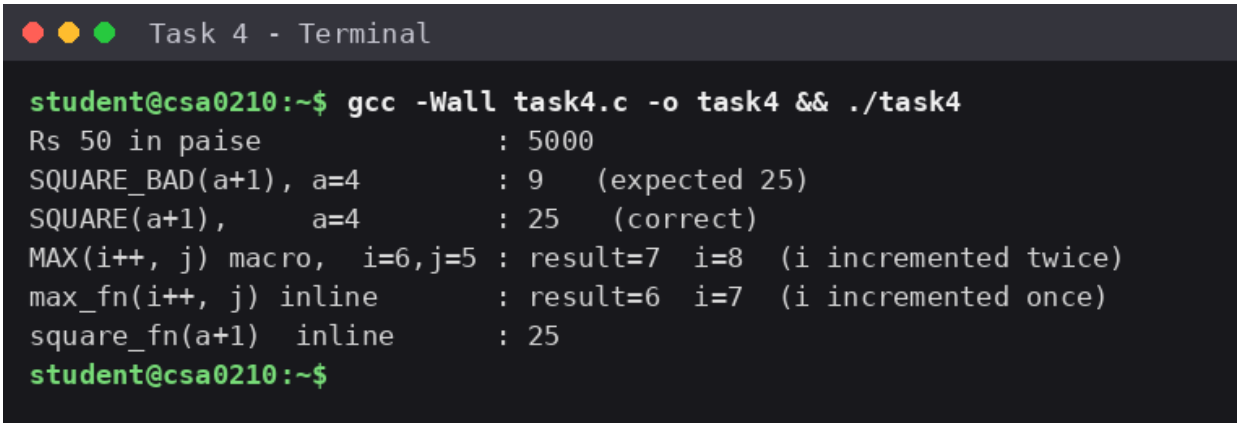


Figure 4.1 — Program output of task4.c.

Task 4 - gcc -E output

```
student@csa0210:~$ gcc -E -P task4.c | sed -n '/int main/,$p'
int main(void)
{
    int a = 4;
    printf("Rs 50 in paise              : %ld\n", ((50) * 100L));
    printf("SQUARE_BAD(a+1), a=4        : %d   (expected 25)\n", a+1 * a+1);
    printf("SQUARE(a+1),     a=4        : %d   (correct)\n", ((a+1) * (a+1)));
    int i = 6, j = 5;
    int m = (((i++) > (j)) ? (i++) : (j));
    printf("MAX(i++, j) macro,  i=6,j=5 : result=%d  i=%d  (i incremented twice)\n", m, i);
    i = 6; j = 5;
    m = max_fn(i++, j);
    printf("max_fn(i++, j) inline       : result=%d  i=%d  (i incremented once)\n", m, i);
student@csa0210:~$
```

Figure 4.2 — Macro expansion shown by gcc -E (preprocessor output of main).

Analysis / Discussion

Expansion with gcc -E. Figure 4.2 shows that RUPEES_TO_PAISE(50) becomes ((50) * 100L) and SQUARE(a+1) becomes ((a+1) * (a+1)); SQUARE_BAD(a+1) becomes a+1 * a+1, which the compiler reads as 4 + (1 × 4) + 1 = 9 instead of 25, matching the printed result. MAX(i++, j) becomes (((i++) > (j)) ? (i++) : (j)).

Double evaluation. With i = 6 and j = 5 the condition i++ > j is true and i becomes 7; then the second i++ yields 7 and i becomes 8. The macro therefore returns 7 and leaves i = 8, but the expected answer is 6 with i = 7. The inline function max_fn(i++, j) evaluates its argument once and gives 6 with i = 7. Parentheses fix the precedence problem but cannot fix double evaluation; only a function (or avoiding side effects in arguments) can.

Pitfall	Cause	Observed	Fix
SQUARE_BAD(a+1)	No parentheses around x	9 instead of 25	Parenthesise: ((x) * (x))
MAX(i++, j)	Argument text appears twice	result 7, i = 8 (should be 6, i = 7)	Use a function / static inline; no side effects in macro arguments
Unparenthesised body	Operator precedence at the call site	Wrong result inside larger expressions	Wrap the whole body in parentheses

Alternative concept — static inline functions instead of macros. square_fn(int) and max_fn(int, int) replace the macros. Each argument is evaluated once, the types are checked by the compiler, and the functions can be stepped through in a debugger. The cost is that each function is tied to one type (int here) whereas a macro works with any type.

Aspect	Macro	static inline function
Time	Expanded in place: no call, O(1)	With optimisation the compiler inlines it, so the same O(1); without inlining a call adds a small constant overhead (push, call, return), still O(1)
Space when used k times	Body is copied at every use: code grows O(k)	May be inlined (also O(k) copies) or kept as one function with k short calls; the compiler chooses, e.g. -Os favours size
Argument evaluation	Text pasted: may evaluate more than once	Exactly once
Type safety	None	Checked against the parameter types

Result

Fully parenthesised macros were written and the gcc -E output proved their expansion. The two pitfalls were demonstrated: SQUARE_BAD(a+1) gave 9 instead of 25, and MAX(i++, j) incremented i twice. Static inline functions removed the double evaluation and add type checking with the same O(1) running time.

Test Cases / Validation Table

Record different inputs, expected results, actual results, and whether each test passed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	Task 2: consumption(1200, 1530), normal reading	330 units	330 units	Pass
2	Task 2: consumption(65400, 120), register rolls over	256 units	256 units	Pass
3	Task 3: compute_bill(0) – no usage, fixed charge only	Rs 50.00	Rs 50.00	Pass
4	Task 3: compute_bill(150) – two slabs used	Rs 162.50	Rs 162.50	Pass
5	Task 3: compute_bill(350) – all slabs used	Rs 950.00	Rs 950.00	Pass
6	Task 3: RATE2_PAISE changed to 230, compute_bill(150)	Rs 165.00	Rs 165.00	Pass
7	Task 4: SQUARE(a+1), a = 4 (parenthesized macro)	25	25	Pass
8	Task 4: SQUARE_BAD(a+1), a = 4 (no parentheses)	25	9	Defect shown (fixed by SQUARE)
9	Task 4: MAX(i++, j), i = 6, j = 5	result 6, i = 7	result 7, i = 8	Defect shown (fixed by max_fn)
10	Task 2: 4,000 bills of Rs 123.45 added as float	Rs 493,800.00	Rs 493,781.2188	Fails – error Rs 18.78
11	Task 2: 4,000 bills of Rs 123.45 added as long paise	Rs 493,800.00	Rs 493,800.00	Pass

Final Submission Checklist

- Student and course details completed
- Task statement, aim, theory, and algorithm included
- Program code is readable and runs successfully
- Output screenshot included
- Analysis and result written in own words
- Test cases completed where applicable
- GitHub repository link added if required